

Tokenization:

Method of Extraction: The architectural data for the ten specified models was extracted programmatically using the Python transformers library. By downloading and parsing the AutoConfig object directly from the Hugging Face Model Hub, the data reflects the exact instantiation parameters used in code rather than potentially outdated or incomplete text in model cards. Authentication via a Hugging Face access token was required and utilized to bypass gating for the Meta Llama 3.1 model. Additionally, we filled in what was missing and could be found in the raw JSON file, also extracted with a Python script.

Uncertainties, Missing Fields, and Disagreements During extraction: the Mixture of Experts details field returned NA for nine out of the ten models. This is not an extraction error or missing data, but rather an architectural reality: only DeepSeek-V3 utilizes an MoE architecture, while the others are standard dense models. Additionally, there was initial uncertainty regarding position encodings. While all models use Rotary Position Embedding, the exact parameters are sometimes nested deeply within scaling dictionaries or implied by the model family type rather than being explicitly defined in top-level configuration keys, requiring fallback logic to confirm, but in the end, they all agreed to use RoPE.

Summary of Main Architectural Differences: The models diverge significantly in vocabulary size and context window. Vocabulary sizes range from roughly 32,000 tokens for Mistral to over 152,000 for Qwen, reflecting different priorities regarding multilingual support versus embedding matrix efficiency. Context lengths are largely bimodal, with models clustering around either a standard 32,000-token window or pushing for extended 131,000-token capacities. DeepSeek-V3 represents the largest deviation from the group; to decouple its massive parameter count from computational cost, it employs 256 routed experts and 1 shared expert across 61 layers, contrasting sharply with the standard 32-layer dense grid seen in the 7B and 8B models. Conversely, the much smaller SmoLM2-1.7B deviates by omitting Grouped-Query Attention entirely, likely because memory constraints on the key-value cache are less pronounced at that smaller scale.

Reflection, Analysis, and General Trends: An analysis of the architectural parameters reveals a strict industry consensus on foundational mathematical operations. When examining the activation functions across all ten models—spanning developers such as Meta, Microsoft, Mistral, and Alibaba—there is 100% agreement. Every single model's configuration specifies **silu** (Swish) as its activation function. While the underlying code of these models wraps this function in a SwiGLU (Swish-Gated Linear Unit) block to create the feed-forward network, the universal adoption of **silu** has made it the undisputed standard for modern LLMs, entirely replacing older alternatives like ReLU or GELU to improve gradient flow.

Furthermore, strict mathematical ratios govern sizing choices across different developer labs. The representation vector size for a single attention head is almost universally locked at 128, calculated by dividing the hidden size by the number of attention heads. The feed-forward network typically expands the hidden state by a multiple of 3.5 before projecting it back down.

Finally, Grouped-Query Attention is nearly mandatory for the larger models to manage memory footprint, usually maintaining a 4:1 ratio of query heads to key-value heads.

Proposed Architecture: If designing a new 8-billion parameter model optimized for complex reasoning loops and dynamic tool routing, adhering to these established mathematical ratios is ideal. A base architecture of 32 layers and a 4096 hidden size would be appropriate, yielding the standard 128 head dimension. Utilizing Grouped-Query Attention with 32 query heads and 8 key-value heads is critical to keeping memory low when processing extensive context or dependency graphs during agentic workflows. The feed-forward layers would follow the 3.5x expansion rule. A context window of 64,000 tokens provides a practical balance, sufficient for analyzing broad system architectures without degrading attention accuracy in the middle of the prompt. Finally, trimming the vocabulary size to around 100,000 tokens, heavily weighted toward English and programming languages, would optimize the embedding matrix and free up parameters for the active processing layers.

Tokenization Differences

The Selected Text:

"Mrs. Dursley pretended she didn't have a sister, because her sister and her good-for-nothing husband were as unDursleyish as it was possible to be."

1. meta-llama/Llama-3.1-8B-Instruct

```
['Mrs', '.', '_D', 'urs', 'ley', '_pretended', '_she', '_didn', "'t", '_have', '_a', '_sister', ',', '_because', '_her', '_sister', '_and', '_her', '_good', '-for', '-no', 'thing', '_husband', '_were', '_as', '_un', 'D', 'urs', 'ley', 'ish', '_as', '_it', '_was', '_possible', '_to', '_be', '.']
```

2. mistralai/Mistral-7B-Instruct-v0.3

```
['_Mrs', '.', '_D', 'urs', 'ley', '_pret', 'ended', '_she', '_didn', '""', 't', '_have', '_a', '_sister', ',', '_because', '_her', '_sister', '_and', '_her', '_good', '-', 'for', '-', 'n', 'othing', '_husband', '_were', '_as', '_un', 'D', 'urs', 'ley', 'ish', '_as', '_it', '_was', '_possible', '_to', '_be', '.']
```

3. ibm-granite/granite-3.3-8b-instruct

```
['_Mrs', '.', '_D', 'urs', 'ley', '_pretended', '_she', '_didn', "'t", '_have', '_a', '_sister', ',', '_because', '_her', '_sister', '_and', '_her', '_good', '-', 'for', '-', 'nothing', '_husband', '_were', '_as', '_un', 'D', 'urs', 'ley', 'ish', '_as', '_it', '_was', '_possible', '_to', '_be', '.']
```

Description of the Causes of Difference:

The distinct tokenizations of this sentence stem from three primary architectural factors:

1. **Vocabulary Size & Compression Limits:** The most obvious difference is in the word "pretended". Llama (with a massive 128,000-token vocabulary) has enough capacity to store "pretended" as a single token. Mistral, constrained by a much smaller ~32,000-token vocabulary, is forced to ruthlessly fragment less common words, splitting it into ['_pret', 'ended'].
2. **Punctuation Merging Rules:** The tokenizers disagree fundamentally on how to handle punctuation attached to words. In the word "didn't", Llama and Granite merge the apostrophe with the trailing 't' (['_didn', "'t"]), whereas Mistral completely isolates the apostrophe (['_didn', '""', 't']).
3. **Hyphenation and Frequency Bias:** The phrase "good-for-nothing" shatters completely differently across all three due to the frequency of subwords in their specific training data.
 - Granite separates the hyphens entirely: ['_good', '-', 'for', '-', 'nothing'].
 - Llama aggressively merges the hyphens into the trailing words: ['_good', '-for', '-no', 'thing'].

- Mistral fragments the word "nothing" itself: ['_good', '-', 'for', '-', 'n', 'othing'].

Agreement Across Remaining Models: When analyzing the full multi-paragraph excerpt against the remaining 7 tokenizers in the assignment list, zero models agreed **completely**. Because every tokenizer relies on a unique Byte-Pair Encoding (BPE) merge-hierarchy derived from its specific pre-training corpus, no two models will perfectly slice a complex text the same way unless they explicitly share the same tokenizer file. This text is also different from anything that it was trained on, so the names of people might not be parsed in the same way.

Constraint decoding:

Testing the models:

--- mistralai/Mistral-7B-Instruct-v0.3 Response ---

The phrase "all but 9" means to subtract 9 from the original number. So, if the farmer has 17 sheep initially, and 9 of them run away, we subtract 9 from 17 to get the remaining number of sheep:

$$17 - 9 = 8$$

So, there are 8 sheep left.

--- Qwen/Qwen2.5-7B-Instruct Response ---

The farmer is left with 9 sheep. The phrase "all but 9 run away" means that every sheep ran away except for 9 of them. So, if the farmer originally had 17 sheep and all but 9 ran away, then 9 sheep remained.

this was the question

"A farmer has 17 sheep. All but 9 run away. How many sheep are left? Explain your reasoning briefly.",

Our strategy:

The strategy for identifying Hebrew-related tokens in the Qwen and Mistral vocabularies involved processing the raw tokenizer JSON files directly to avoid massive memory overhead. First, we normalized the tokens by replacing model-specific space markers, such as the Byte-Pair Encoding marker used by Qwen and the SentencePiece marker used by Mistral, with standard spaces to ensure valid tokens were not falsely rejected. Next, we applied a strict

whitelist using a regular expression that evaluated the entire token string from start to finish. A token was only accepted if it consisted exclusively of standard Hebrew characters, alphabetic presentation forms, numbers, and standard ASCII punctuation or whitespace. If a token contained even a single character outside this set, such as a Latin or Cyrillic letter, it was discarded. Finally, this strict whitelist approach inherently filtered out raw byte fallback tokens and structural control tokens. Because raw byte tokens are represented in the vocabulary as strings containing angle brackets and Latin letters, they naturally failed the regular expression check. This deliberate exclusion was necessary because allowing raw bytes to pass would inevitably introduce tokens used by other multi-byte languages, violating the requirement to strictly isolate Hebrew text components.

Constrained decoding was implemented using the Hugging Face transformers library by creating a custom LogitsProcessor. Specifically, we defined a class named `AllowedTokensLogitsProcessor` that intercepts the model's vocabulary predictions (logits) at every generation step. When initialized, it creates a boolean mask over the entire vocabulary size, setting the indices of the allowed Hebrew tokens (and the EOS token) to True. During generation, the processor's `__call__` method applies this mask to the prediction scores, setting the logits of all disallowed tokens to negative infinity (-inf). This custom processor is then passed to the `model.generate()` method via the `logits_processor` argument. As a result, when the model applies the softmax function to choose the next token, all non-Hebrew tokens receive a probability of zero, mathematically constraining the model to only output the predefined allowed tokens.

FineTuning:

We fine-tuned Qwen to produce Hebrew answers using LoRA rather than full-model training. The main training script is `finetune_qwen.py`. It loads the base model and tokenizer for Qwen, makes sure a pad token exists, and then applies a LoRA adapter on the attention projection layers `q_proj`, `k_proj`, `v_proj`, and `o_proj`. The LoRA setup uses rank 16, alpha 32, and dropout 0.05, which keeps the number of trainable parameters small while still allowing the model to adapt to the new task.

For training data, the script reads `full_hebrew_sft_data.jsonl`. Each example contains a `messages` field in chat format. Before training, the script converts each example into the model's chat template so the data matches the way the model is expected to be prompted at inference time. This matters because the model is instruction-tuned, so the training examples need to be formatted as conversations rather than plain text.

We trained in two modes. The normal mode runs the full dataset for one epoch with batch size 2, gradient accumulation 4, learning rate $2e-4$, max sequence length 512, and bf16 enabled. The output adapter is saved to `qwen-hebrew-lora-final`. The script also includes an overfit

sanity-check mode, where it trains on only two examples for many epochs and logs every step. That mode is there to verify that the training loop works and that the model can easily memorize a tiny dataset before doing the full run. We didn't include the overfitting script because it didn't seem relevant.

To show that fine-tuning worked, we evaluated the base model and the fine-tuned model on a 20-item benchmark. The comparison output was collected in `eval_outputs.jsonl`, and then post-processed with `evaluate_from_json.py`. That evaluation script measures how many non-Hebrew characters appear in the base and fine-tuned outputs, and also calculates the percentage of non-Hebrew characters relative to total non-space characters. It then computes the difference between the two models and writes a markdown table plus a JSONL summary.

The evaluation results show a clear shift after fine-tuning. Across all 20 benchmark prompts, the fine-tuned model reduced the non-Hebrew character count compared with the base model every time. The average non-Hebrew percentage dropped from 100.00% in the base outputs to about 11.57% in the fine-tuned outputs, with an average delta of about -88.43 percentage points. In other words, the model moved strongly away from English output and toward Hebrew output after fine-tuning. The qualitative examples also show the same trend: before fine-tuning, the model answered naturally in English, while after fine-tuning, it started producing Hebrew text.

The main conclusion is that the LoRA fine-tuning changed the model behavior in the intended direction. It did not make every output perfect Hebrew, but it clearly pushed the model toward Hebrew generation, which is what the evaluation was designed to demonstrate.